

PHASE 4 — LIMITATIONS

Consolidated from every model's registry entry. This document exists so limitations are never buried inside individual scripts — per Master Prompt Section 16 (Documentation must cover “Limitations” as a first-class topic).

1. Synthetic data, not a real market

Every Phase 4 model is trained on the Phase 1 synthetic dataset (seed=42, 8,000 properties). The dataset was designed with realistic business logic (better locality → higher price, oversupply → longer days-on-market, etc.), but no metric in this phase (R^2 , MAE, ROC-AUC, or any other) should be interpreted as a claim about real-world real-estate accuracy. All metrics describe how well each model learned the *generator's* logic.

2. The transaction-price / market-price compounding bug (carried forward from Phase 3 EDA)

Phase 3 EDA traced a real bug in the data generator: the locality price-growth formula compounds monthly with **no ceiling or mean-reversion term**. This inflates `transactions.transaction_price`, `transactions.price_per_sqft`, and `market_monthly.average_price_sqft` for high-demand localities over the 2020–2025 window (one locality: ₹13,592/sqft → ₹1,923,519/sqft).

- **Models A and B (valuation, rent)** are unaffected — they use `properties.asking_price` and `properties.monthly_rent`, which are point-in-time attributes, not the compounding series.
- **Model C (price forecasting)** targets `market_monthly.average_price_sqft` directly and therefore inherits this bug. Per Master Prompt Section 4.6, the preferred fix (“regenerate data at source”) was not executable — **the original generator script was not included in any of the three delivered Phase 1/2/3 zip files**, only its output CSVs. The documented fallback (log1p-space modeling, honest out-of-time evaluation) was applied instead, and the model's status is forced to **CONDITIONAL** regardless of its strong log-space metrics.
- **Model G (risk score)** deliberately treats locality price volatility (which is elevated in the same affected localities) as a genuine risk signal, on the reasoning that a buyer cannot rely on gains that came from a data artifact any more than from real volatility — but this means the `price_volatility_risk` component inherits the same caveat.
- **Recommendation for a future phase:** regenerate Phase 1 data with a growth ceiling / mean-reversion term in the price-growth formula, rerun Phase 3 EDA, and retrain Model C (and re-check Model G's volatility component) before any of these are used for anything beyond directional guidance.

3. Two models are REJECTED, by design

`dom_v1_gbr` (days-on-market) and `sale_probability_v1_gbr` (sale outcome) were both trained, evaluated with a proper held-out split, and found to carry **no learnable signal** beyond a naive baseline

($R^2 \approx -0.01$ and ROC-AUC ≈ 0.52 respectively). This independently confirms what Phase 3 EDA had already flagged for days-on-market. Per Master Prompt Section 4.2/4.5, both are REJECTED and must not be consumed by Phase 9's Prediction Agent. This is treated as a correct, expected governance outcome — publishing a near-random prediction as if it were signal would be a worse failure than rejecting the model.

4. Model D does not beat its baseline

demand_forecast_v1_gbr_6m achieves $R^2=0.932$, which looks strong in isolation, but a simple persistence baseline (assume demand in 6 months equals demand today) achieves $R^2=0.933$ — marginally better. Demand is highly autocorrelated month-to-month in this dataset, so “no change” is already a strong forecast. The model is registered as **CONDITIONAL**, not PASS or PASS WITH LIMITATIONS, because Master Prompt Section 4.5 requires a “useful baseline comparison” for promotion — a high absolute R^2 alone is not sufficient when a trivial baseline matches or beats it. Phase 9's Prediction Agent must treat this model as requiring human review before it is used for anything beyond a directional, clearly-caveated signal.

5. Cross-target leakage guard between Models A and B

monthly_rent correlates strongly with asking_price (Phase 3 EDA: $r=0.824$) because both are driven by largely the same underlying factors (mainly area_sqft). Including each model's counterpart as a feature would let each “cheat” by reading off a value that, in a real deployment, is often unknown at the point you want the *other* prediction (e.g., you rarely have a confirmed market rent for a property whose value you are trying to estimate). Both models are therefore trained independently from structural and locational features only. This is a deliberate, documented design choice, not an oversight — enforced by an automated test (test_valuation_model_excludes_rent_feature, test_rent_model_excludes_price_feature).

6. Model G (risk score) is a composite, not a fitted model

Weights (25/20/20/20/15%) are business judgment, explicitly documented, not statistically fitted. There is no ground-truth “was this actually a risky investment” label in a synthetic dataset to calibrate against, so the model's status is UNCALIBRATED. Its output must be used as a **relative ranking** across properties in this dataset, never presented to an end user as a calibrated probability of loss, until it can be backtested against real historical outcomes.

7. No live database dependency in this phase

Phase 4 reads directly from Phase 1's data/processed/*.csv files rather than requiring a live PostgreSQL connection, because a running real_estate_db instance was not part of the environment available when this phase was built. The join/feature logic in src/data_loader.py is identical either way — pointing it at a live database instead of CSVs (e.g., via psycopg2, following Phase 1/3's connection.py pattern) would require no changes to any of the seven model scripts.

8. Missing-value handling

Numeric feature gaps are filled with the column median; categorical gaps are filled with the literal string "Unknown" (which then becomes its own one-hot category, so the model can learn “unknown is different from known,” rather than being silently merged into an arbitrary existing category). Rows with a missing **target** (not feature) are dropped from training and evaluation, never imputed — imputing a target would mean training the model to predict its own imputation rule.